



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

**Практическая работа №2
Система хранения данных**

Студент группы ИНБО-12-23, Албахтин И.В.

(подпись)

Преподаватель Воронцов Ю.А.

(подпись)

Москва 2026 г.

Содержание

1 Цель и выполненные задания	3
2 Виртуальная инфраструктура	3
3 Схема и импорт данных.....	6
4 Аналитические SQL-задачи	6
4.1 Топ товарных категорий по выручке	6
4.2 Поведение покупателей при оплате	7
4.3 Оценка в зависимости от срока доставки.....	8
4.4 Рейтинг продавцов по заказам	9
5 Grafana и проверка мониторинга.....	9
6 Выводы	10
Ответы на контрольные вопросы	11
Источники и приложения	13

1 Цель и выполненные задания

Цель работы — развернуть серверное окружение, подготовить схему и импортировать взаимосвязанные CSV, выполнить четыре аналитические SQL-задачи и собрать дашборд Grafana. Все сервисы и данные созданы заново в отдельной виртуальной машине для Албахтина И.В.; результаты запросов сохранены после фактического выполнения.

2 Виртуальная инфраструктура

Результаты приведены в таблице 1.

Таблица 1 – Конфигурация стенда

Параметр	Значение
Имя VM	TIABD-Albakhtin-IV
UUID VM	40603a5d-2404-4561-a4d6-02d13a8b388d
ОС	Ubuntu Server 22.04.5 LTS
Ресурсы	2 vCPU, 4 ГБ RAM, VDI 40 ГБ
Пользователь / hostname	albakhtin / tiabd-albakhtin-iv
Сеть	NAT, 10.0.2.15; SSH через 127.0.0.1:2223
PostgreSQL	PostgreSQL 16.15 (Debian 16.15-1.pgdg13+2)

На рисунке 1 представлен результат анализа.

```
TIABD-Albakhtin-IV | student: Albakhtin I.V. | fresh run

HOSTNAME: tiabd-albakhtin-iv
USER: albakhtin
Ubuntu 22.04.5 LTS
Mem:          total        used        free        shared  buff/cache   available
Swap:          0B           0B           0B          164Mi     635Mi        2.1Gi

NAME      SIZE TYPE MOUNTPOINT
loop0     87M loop /snap/lxd/29351
loop1    38.8M loop /snap/snapd/21759
loop2    63.9M loop /snap/core20/2318
sda       40G disk
└─sda1     1M part
└─sda2    40G part /
sr0      1024M rom

VM UUID: 40603a5d-2404-4561-a4d6-02d13a8b388d
```

Рисунок 1 – Реальная консоль отдельной VM с параметрами стенда

В Docker Compose развернуты PostgreSQL, ADCM с отдельной служебной БД, postgres_exporter, Prometheus и Grafana. PostgreSQL использует официальный образ postgres:16. В ADCM загружен учебный community-бандл с действиями реальной проверки TCP-порта PostgreSQL и HTTP-доступности мониторинга. Промышленный бинарный дистрибутив Arenadata PostgreSQL в этом стенде не установлен.

Результаты приведены в таблице 2.

Таблица 2 – Доступ к сервисам с Windows

Сервис	Локальный адрес
ADCM	http://127.0.0.1:8001
PostgreSQL	127.0.0.1:5433
Prometheus	http://127.0.0.1:9091
Grafana	http://127.0.0.1:11211
JupyterLab	http://127.0.0.1:8889
Spark UI во время работы	http://127.0.0.1:4041

На рисунке 2 представлен результат анализа.

```
TIABD-Albakhtin-IV | student: Albakhtin I.V. | fresh run

NEW DOCKER SERVICES
adcm                running
adcm-db             running    healthy
adpg                running    healthy
grafana             running
postgres-exporter   running
prometheus          running

TABLE COUNTS
customers            99441
geolocation          1000163
leads_closed         842
leads_qualified       8000
order_items          112650
order_payments       103886
order_reviews        100000
orders               99441
product_category_name_translation  73
products             32951
sellers              3095
Foreign keys validated: 9
Grafana datasource: OK
pg_up: 1
```

Рисунок 2 – Новые контейнеры и успешные проверки PostgreSQL и мониторинга

На рисунке 3 представлен результат анализа.

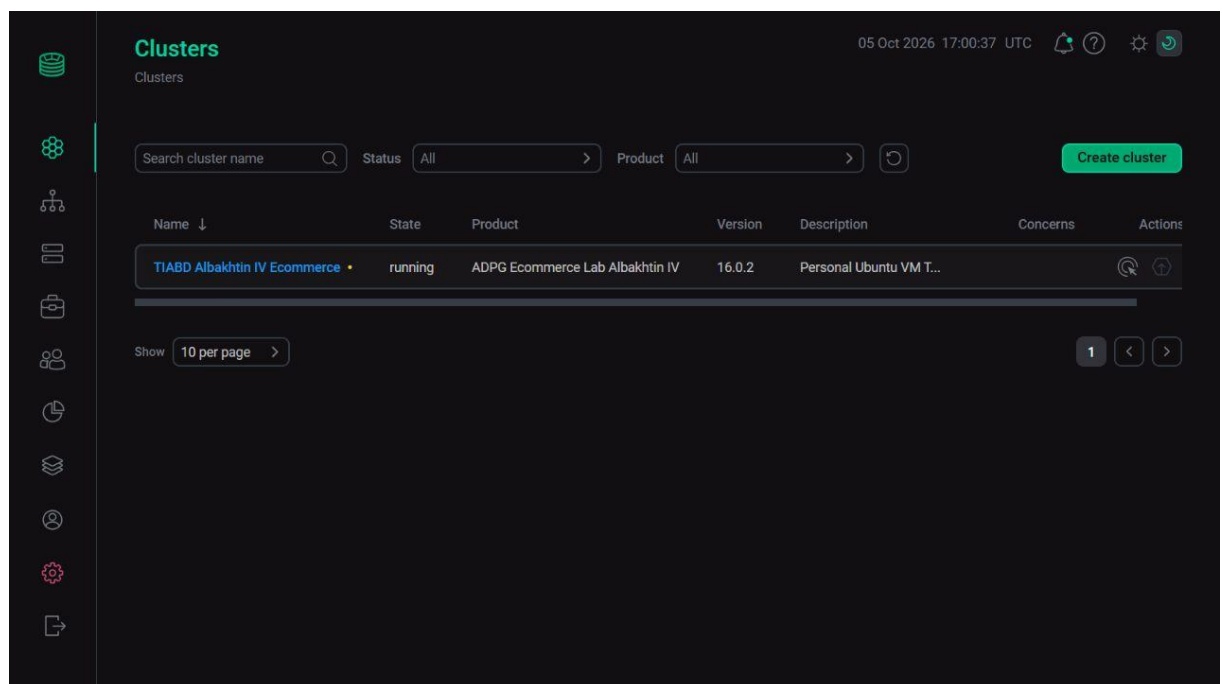


Рисунок 3 – Личный кластер учебного бандла в ADCM

3 Схема и импорт данных

Исходные 11 CSV Olist сохранены без изменения содержимого; контрольные суммы приведены в source_inputs.json. Таблицы, первичные и внешние ключи созданы до импорта. Зависимые таблицы загружаются после справочников. Для отсутствующих переводов добавлены две категории с NULL-переводом. В leads_closed ссылки на отсутствующих продавцов заменены на NULL в промежуточной таблице; исходный CSV не изменён.

```
docker compose exec -T adpg psql -U postgres -d ecommerce -f /sql/schema.sql
docker compose exec -T adpg psql -U postgres -d ecommerce -f /sql/load.sql
docker compose exec -T adpg psql -U postgres -d ecommerce -f /sql/analytics.sql
```

Результаты приведены в таблице 3.

Таблица 3 – Фактически импортированные строки

Таблица	Строк
customers	99 441
geolocation	1 000 163
leads_closed	842
leads_qualified	8 000
order_items	112 650
order_payments	103 886
order_reviews	100 000
orders	99 441
product_category_name_translation	73
products	32 951
sellers	3 095
Итого	1 560 542

Все 11 таблиц присутствуют в новой базе. Подтверждено 9 внешних ключей с convalidated=true. Импорт использует ON_ERROR_STOP, поэтому SQL-ошибка останавливает выполнение. ANALYZE обновляет статистику для планировщика после загрузки.

4 Аналитические SQL-задачи

4.1 Топ товарных категорий по выручке

-- 1. Top 10 categories by revenue with English translation.

```
SELECT
  p.product_category_name,
  COALESCE(t.product_category_name_english, 'not translated') AS category_en,
  ROUND(SUM(oi.price), 2) AS revenue,
  COUNT(DISTINCT oi.order_id) AS orders_count,
```

```

COUNT(*) AS items_count
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
LEFT JOIN product_category_name_translation t
  ON t.product_category_name = p.product_category_name
GROUP BY p.product_category_name, t.product_category_name_english
ORDER BY revenue DESC
LIMIT 10;

```

Результаты приведены в таблице 4.

Таблица 4 – Топ товарных категорий по выручке

Категория	Выручка R\$	Заказы	Позиции
health_beauty	1 258 681,34	8 836	9 670
watches_gifts	1 205 005,68	5 624	5 991
bed_bath_table	1 036 988,68	9 417	11 115
sports_leisure	988 048,97	7 720	8 641
computers_accessories	911 954,32	6 689	7 827
furniture_decor	729 762,49	6 449	8 334
cool_stuff	635 290,85	3 632	3 796
housewares	632 248,66	5 884	6 964
auto	592 720,11	3 897	4 235
garden_tools	485 256,46	3 518	4 347

Лидирует health_beauty: 1 258 681,34 R\$. SUM(price) отражает стоимость товарных позиций и не включает доставку. COUNT(DISTINCT order_id) считает заказы независимо от числа позиций.

4.2 Поведение покупателей при оплате

-- 2. Payment behaviour.

```

SELECT
  payment_type,
  COUNT(*) AS payments_count,
  ROUND(AVG(payment_installments), 2) AS avg_installments,
  ROUND(AVG(payment_value), 2) AS avg_payment,
  ROUND(100.0 * COUNT(DISTINCT order_id)
    / (SELECT COUNT(DISTINCT order_id) FROM order_payments), 2) AS order_share_pct
FROM order_payments
GROUP BY payment_type
ORDER BY payments_count DESC;

```

Результаты приведены в таблице 5.

Таблица 5 – Поведение покупателей при оплате

Тип	Платежи	Части	Средний R\$	Заказы %
credit_card	76 795	3,51	163,32	76,94
boleto	19 784	1,00	145,03	19,90
voucher	5 775	1,00	65,70	3,89
debit_card	1 529	1,00	142,57	1,54
not defined	3	1,00	0,00	0,00

Наиболее распространена оплата credit_card: 76 795 платёжных строк. Доли рассчитаны по уникальным заказам; они могут суммироваться более чем в 100%, если заказ оплачен несколькими способами.

4.3 Оценка в зависимости от срока доставки

-- 3. Review score depending on delivery speed.

```
WITH reviews AS (
  SELECT order_id, AVG(review_score) AS review_score
  FROM order_reviews
  GROUP BY order_id
), delivery AS (
  SELECT
    o.order_id,
    CASE
      WHEN o.order_delivered_customer_date::date - o.order_purchase_timestamp::date <= 5
      THEN 'up to 5 days'
      WHEN o.order_delivered_customer_date::date - o.order_purchase_timestamp::date <= 10
      THEN '6-10 days'
      ELSE 'more than 10 days'
    END AS delivery_group,
    CASE
      WHEN o.order_delivered_customer_date::date - o.order_purchase_timestamp::date <= 5
      THEN 1
      WHEN o.order_delivered_customer_date::date - o.order_purchase_timestamp::date <= 10
      THEN 2
      ELSE 3
    END AS sort_order,
    r.review_score
  FROM orders o
  JOIN reviews r ON r.order_id = o.order_id
  WHERE o.order_delivered_customer_date IS NOT NULL
)
SELECT delivery_group,
  COUNT(DISTINCT order_id) AS orders_count,
  ROUND(AVG(review_score), 2) AS avg_review_score
FROM delivery
GROUP BY delivery_group, sort_order
ORDER BY sort_order;
```

Результаты приведены в таблице 6.

Таблица 6 – Оценка в зависимости от срока доставки

Интервал	Заказы	Средняя оценка
up to 5 days	16 707	4,43
6-10 days	33 161	4,34
more than 10 days	46 608	3,90

Отзывы предварительно усреднены до одной строки на заказ, поэтому заказ с несколькими отзывами не получает дополнительный вес. Средняя

оценка снижается при росте срока доставки. Это связь в наблюдениях, а не доказательство причинности; недоставленные заказы исключены из данной задачи.

4.4 Рейтинг продавцов по заказам

-- 4. Top 10 sellers by number of orders.

```
SELECT
  oi.seller_id,
  COUNT(DISTINCT oi.order_id) AS orders_count,
  COUNT(DISTINCT c.customer_unique_id) AS customers_count,
  COUNT(*) AS items_count,
  ROUND(SUM(oi.price), 2) AS revenue,
  ROUND(AVG(oi.freight_value), 2) AS avg_freight
FROM order_items oi
JOIN orders o ON o.order_id = oi.order_id
JOIN customers c ON c.customer_id = o.customer_id
GROUP BY oi.seller_id
ORDER BY orders_count DESC, revenue DESC
LIMIT 10;
```

Результаты приведены в таблице 7.

Таблица 7 – Рейтинг продавцов по заказам

Продавец	Заказы	Клиенты	Позиции	Оборот R\$
6560211a...	1 854	1 824	2 033	123 304,83
4a3ca931...	1 806	1 792	1 987	200 472,92
cc419e06...	1 706	1 657	1 775	104 288,42
1f50f920...	1 404	1 388	1 931	106 939,21
da8622b1...	1 314	1 275	1 551	160 236,57
955fee92...	1 287	1 282	1 499	135 171,70
7a67c85e...	1 160	1 155	1 171	141 745,53
ea8482cd...	1 146	1 133	1 203	37 177,52
4869f7a5...	1 132	1 124	1 156	229 472,63
3d871de0...	1 080	1 074	1 147	94 914,20

Рейтинг сортируется по числу уникальных заказов и затем по выручке. Покупатели считаются по customer_unique_id: customer_id в Olist относится к записи покупателя конкретного заказа. Лидер по активности может отличаться от лидера по обороту.

5 Grafana и проверка мониторинга

PostgreSQL datasource ecommerce-postgres подключён к новой базе. Проверка datasource health вернула ОК, база Grafana — ok; значение pg_up в Prometheus равно 1. Дашборд содержит четыре обязательные панели и две

дополнительные: динамику заказов, распределение оценок, способы оплаты, топ категорий, общую выручку и число уникальных покупателей.

На рисунке 4 представлен результат анализа.

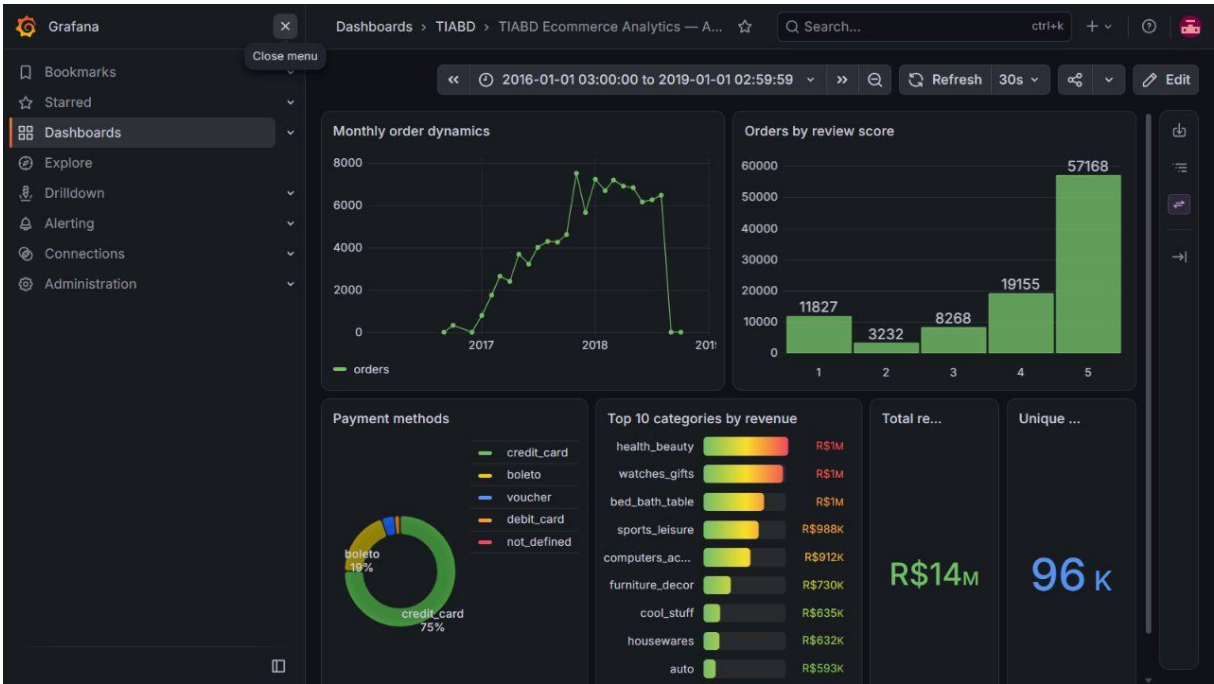


Рисунок 4 – Фактический дашборд из шести панелей в новой VM
Результаты приведены в таблице 8.

Таблица 8 – Контрольные показатели дашборда

Показатель	Значение
Стоимость товарных позиций	13 591 643,70 R\$
Уникальные покупатели	96 096
PostgreSQL exporter pg_up	1
Grafana datasource health	OK

6 Выводы

1. Создана независимая VM и заново развернут полный учебный контур хранения, управления и мониторинга.
2. Порядок импорта и промежуточные таблицы позволили сохранить строки при проверке типов и ссылочной целостности.
3. Четыре аналитические задачи используют корректную детализацию: позиции для выручки, уникальные заказы для активности и один средний отзыв на заказ.
4. Рост срока доставки сопровождается снижением оценки, но имеющиеся наблюдения не устанавливают причинность.

5. Дашборд и проверки health/pg_up подтверждают доступность новой базы для анализа и мониторинга.

Ограничения: локальная учебная VM не моделирует отказоустойчивый промышленный кластер; официальный PostgreSQL и community-бандл ADCM не являются промышленной сборкой ADPG.

Ответы на контрольные вопросы

1. Почему создание таблиц должно выполняться до импорта данных из CSV-файлов?

Импорт выполняется в уже существующий объект с определёнными столбцами и типами. Предварительное создание таблиц позволяет задать ключи, ограничения и порядок загрузки, а СУБД может сразу проверять корректность данных.

2. Какие проблемы могут возникнуть, если структура CSV-файла не соответствует структуре целевой таблицы?

Возможны смещение значений по столбцам, ошибки преобразования типов, нарушение ограничений NOT NULL и потеря части данных. Даже успешная загрузка может дать логически неверные значения, если порядок столбцов не совпал.

3. Что произойдёт при попытке загрузить строку с внешним ключом без записи в родительской таблице?

При активном ограничении FOREIGN KEY PostgreSQL отклонит строку и сообщит о нарушении ссылочной целостности. Поэтому родительские справочники загружаются раньше дочерних таблиц либо проблемная ссылка предварительно нормализуется.

4. Почему перед импортом взаимосвязанных таблиц необходимо определить зависимости между ними?

Зависимости задают правильную последовательность загрузки: сначала таблицы, на которые ссылаются, затем таблицы с внешними ключами. Это предотвращает ошибки и упрощает повторный запуск импорта.

5. Для чего используется GROUP BY при решении аналитических задач?

GROUP BY объединяет строки с одинаковыми значениями выбранных признаков и позволяет вычислять агрегаты для каждой группы. В работе он использован для категорий, способов оплаты, интервалов доставки и продавцов.

6. Почему иногда нужен COUNT(DISTINCT order_id), а не COUNT(*)?

COUNT(*) считает строки результата, а один заказ может содержать несколько позиций, платежей или отзывов. COUNT(DISTINCT order_id) устраняет такое размножение и возвращает число уникальных заказов.

7. Для чего используется ORDER BY ... DESC при построении рейтингов?

DESC сортирует значения от большего к меньшему. После такой сортировки LIMIT 10 выбирает десять лидеров по заданной метрике.

8. Что означает источник данных в Grafana и что указывается для PostgreSQL?

Источник данных — настроенное соединение, через которое Grafana выполняет запросы. Для PostgreSQL указываются адрес и порт сервера, база данных, пользователь, пароль, режим TLS и параметры версии/совместимости.

9. Почему для динамики заказов подходит линейный график, а не круговая диаграмма?

Линейный график сохраняет хронологический порядок и показывает тренд, пики и сезонность. Круговая диаграмма сравнивает доли категорий и не передаёт последовательность изменения во времени.

10. Чем преобразование данных в Grafana отличается от преобразования в SQL?

SQL-преобразование выполняется на стороне базы до передачи результата и обычно лучше масштабируется. Преобразования Grafana применяются к уже полученным наборам и удобны для объединения, переименования и подготовки полей конкретной панели.

11. Что может означать отсутствие данных на панели при исправном подключении?

Причинами могут быть неверный временной диапазон, фильтры, ошибка в SQL, отсутствие прав на таблицу, NULL в поле времени или несовпадение формата результата с выбранным типом панели. Проверку начинают с Query inspector и запуска запроса без ограничений.

12. Почему визуализация должна сопровождаться текстовой интерпретацией?

График показывает форму данных, но не фиксирует главный вывод, ограничения и возможные причины. Текст связывает наблюдение с задачей, уточняет единицы измерения и не позволяет принять корреляцию за доказанную причинность.

Источники и приложения

Источники: предоставленная методичка №2 и набор Olist. SQL, схема, Docker Compose и бандл ADCM — Исходники/ПР2_Хранение; измерения и скриншоты — Результаты/Хранение.